Operating Systems | CPU Scheduling

Academic Article · Technical Report

The Longest Job First (LJF) CPU Scheduling Algorithm

Concepts, Variants, Worked Examples, Mathematical Analysis and Implementations

A comprehensive and educational study in operating systems and scheduling theory

Student name:
Sadegh Azodi

Abstract

This article presents a comprehensive study of the Longest Job First (LJF) CPU scheduling algorithm, which dispatches the ready process with the largest CPU burst time. After reviewing processes, scheduling criteria and the position of LJF among scheduling policies, the article defines the non-preemptive algorithm, distinguishes it carefully from its preemptive relatives, including Longest Remaining Time First (LRTF), and details its execution procedure with a flowchart. Five fully solved examples (simultaneous arrivals, staggered arrivals, equal burst times, a ten-process workload with ready-queue analysis, and a preemptive comparison) are accompanied by Gantt charts and by waiting, turnaround and response times. A mathematical analysis shows, by an exchange argument and exhaustive verification, that LJF maximises the average waiting time when all processes arrive together, while longest-first ordering remains a sound heuristic for makespan on parallel machines. The article compares LJF with FCFS, SJF, SRTF, Round Robin, priority scheduling and LRTF, provides tested implementations in Python, C++ and Java, discusses applications and limitations, and reviews improvements such as aging and hybrid policies.

Keywords: CPU scheduling; Longest Job First; LJF; LRTF; non-preemptive scheduling; waiting time; turnaround time; starvation; Gantt chart

Table of Contents

Introduction **7**
Chapter 1: Fundamental Concepts **7**
1.1 Introduction to Operating Systems 7
1.2 Definition of a Process 8
1.3 Process Life Cycle 8
1.4 Definition of CPU Scheduling 8
1.5 Importance of CPU Scheduling 8
1.6 Design Goals and Evaluation Criteria 9
1.7 Position of LJF among Scheduling Algorithms 9
Chapter 2: The LJF Algorithm **10**
2.1 Precise Definition 10
2.2 History and Context 10
2.3 Core Idea and Logic 10
2.4 How the Next Process Is Selected 10
2.5 Entry of Processes into the Ready Queue 11
2.6 Managing the Process Queue 11
2.7 The Role of Burst Time 11
2.8 Arrival Time versus Execution Time 11
2.9 Behaviour under Different Conditions 11
Chapter 3: Types of LJF **12**
3.1 Non-Preemptive LJF 12
3.2 Preemptive LJF 12
3.3 The Concept of Preemption 12
3.4 Preemptive versus Non-Preemptive 13
3.5 When the Running Process Changes 13
3.6 Advantages and Drawbacks of Each Version 13
3.7 Independent Examples for Each Version 13
3.8 Related Terminology: LJF versus LRTF 14
Chapter 4: Operation and Execution Steps **14**
4.1 Step-by-Step Procedure 14
4.2 Flowchart 15
4.3 Pseudo-code 15
Chapter 5: Numerical Examples and Step-by-Step Solutions **15**
5.1 Example 1: All Processes Arrive at Time 0 16
5.2 Example 2: Different Arrival Times 17
5.3 Example 3: Equal Burst Times 19
5.4 Example 4: Ten Processes and Ready-Queue Analysis 20
5.5 Example 5: Preemptive Version (LRTF) versus Non-Preemptive LJF 22
Chapter 6: Formulas and Mathematical Analysis **23**
6.1 Turnaround Time 23
6.2 Waiting Time 23
6.3 Average Waiting Time 24
6.4 Average Turnaround Time 24
6.5 Response Time 24
6.6 CPU Utilisation and Throughput 24
6.7 Optimality Analysis: Why LJF Maximises the Average Waiting Time 25
6.8 Where Longest-First Is Good: Makespan on Parallel Machines 26
6.9 Time and Space Complexity 26
Chapter 7: Diagrams and Educational Figures **27**
Chapter 8: Advantages and Disadvantages of LJF **29**
8.1 Advantages 29
8.2 Disadvantages 30
Chapter 9: Comparison with Other Algorithms **31**
9.1 Qualitative Comparison 31
9.2 Behaviour under Different Conditions 32
9.3 Numerical Comparison 32
Chapter 10: Programming Implementations **32**
10.1 Python 33
10.2 C++ 34

10.3 Java 35
10.4 Sample Input and Output 36
10.5 Boundary Cases and Verification 37
Chapter 11: Applications and Case Studies 38
11.1 Theoretical Use versus Practice in Real Operating Systems 38
11.2 Batch Processing and Offline Systems 38
11.3 Educational Simulation Environments 38
11.4 Environments with Long-Running Tasks 39
Chapter 12: Improvements and Alternative Approaches 39
12.1 Aging 39
12.2 Starvation Prevention 39
12.3 Hybrid Scheduling 39
12.4 Dynamic Priority 40
12.5 Combining LJF and Priority Scheduling 40
12.6 Better Handling of Short Processes 40
12.7 Fairness-Oriented Alternatives 40
Chapter 13: Summary and Conclusions 40
13.1 Review of Main Concepts 40
13.2 Summary of the Algorithm and Numerical Findings 41
13.3 Evaluation of Strengths and Weaknesses 41
13.4 Comparison with Other Algorithms 41
13.5 Limitations of the Algorithm and of This Study 41
13.6 Suggestions for Future Research 41
Chapter 14: References 41

List of Tables

Table 1: Basic concepts used in this article 9
Table 2: Comparison of non-preemptive LJF, LJF-PA and LRTF 13
Table 3: Results of the three variants on the Example 5 workload 14
Table 4: Process specifications of Example 1 16
Table 5: Results of Example 1 under LJF 16
Table 6: Process specifications of Example 2 17
Table 7: Completion time calculation for Example 2 18
Table 8: Turnaround time calculation for Example 2 18
Table 9: Waiting time calculation for Example 2 18
Table 10: Summary of results of Example 2 18
Table 11: Process specifications of Example 3 19
Table 12: Results of Example 3 under LJF 20
Table 13: Process specifications of Example 4 20
Table 14: Ready queue at each decision instant in Example 4 (ordered by decreasing BT) 21
Table 15: Results of Example 4 under LJF 21
Table 16: Process specifications of Example 5 22
Table 17: Example 5 results: LJF (non-preemptive) 22
Table 18: Example 5 results: LRTF (preemptive) 23
Table 19: Summary of formulas 25
Table 20: Time and space complexity of LJF implementations 26
Table 21: Catalogue of figures 27
Table 22: Advantages and disadvantages of LJF 31
Table 23: Comprehensive comparison of scheduling algorithms 31
Table 24: Average metrics of all algorithms on the Example 2 workload 32
Table 25: Line-by-line explanation of the Python program 34
Table 26: Line-by-line explanation of the C++ program 35
Table 27: Line-by-line explanation of the Java program 36
Table 28: Verification of the implementations on the article's workloads 37
Table 29: Suitability of LJF in different environments 38
Table 30: Methods that complement or replace LJF 40
Table 31: Summary of numerical results (LJF unless stated, with SJF AWT for reference) 41

List of Figures

Figure 1: Process state diagram; the dispatch edge Ready → Running is where LJF makes its decision. 8
Figure 2: Flowchart of non-preemptive LJF, including the idle-CPU branch. 15
Figure 3: Gantt chart of Example 1. 16
Figure 4: Gantt chart of Example 2 (arrows mark arrivals). 17
Figure 5: Gantt chart of Example 3 (equal bursts). 19
Figure 6: Gantt chart of Example 4. 21
Figure 7: Example 5: non-preemptive LJF (top) versus LRTF (bottom). 22
Figure 8: Total waiting time of all 120 orderings of the Example 1 workload. 25
Figure 9: Growth of linear-scan (n^2) versus heap-based (n log n) total selection cost. 26
Figure 10: Execution order of the Example 1 workload under FCFS, SJF and LJF. 27
Figure 11: Ready-queue length at the decision instants of Example 4. 28
Figure 12: Waiting time of each process of Example 1 under FCFS, SJF and LJF. 28
Figure 13: Turnaround time of each process of Example 1 under FCFS, SJF and LJF. 28
Figure 14: Waiting time of Example 2 under LJF and FCFS. 29
Figure 15: Waiting time of Example 2 under LJF and SJF. 29

List of Abbreviations

Abbreviation	Meaning
AT	Arrival Time
ATAT	Average Turnaround Time
AWT	Average Waiting Time
ART	Average Response Time
BT	Burst Time
CPU	Central Processing Unit
CT	Completion Time
FCFS	First Come First Served
HRRN	Highest Response Ratio Next
I/O	Input / Output
LJF	Longest Job First
LJF-PA	LJF, preemptive on arrival (static key)
LPT	Longest Processing Time (rule)
LRTF	Longest Remaining Time First
NP	Non-Preemptive
OS	Operating System
PCB	Process Control Block
PID	Process Identifier
RR	Round Robin
RT	Response Time
SJF	Shortest Job First
SRTF	Shortest Remaining Time First
ST	Start Time (first CPU allocation)
TAT	Turnaround Time
WT	Waiting Time

Introduction

The central processing unit (CPU) is the scarcest resource in a multiprogrammed computer: while one process occupies it, every other runnable process must wait. The component of the operating system that decides which runnable process receives the CPU next is the CPU scheduler, and the rule it follows is called a scheduling algorithm. Because the rule determines how long users wait, how busy the hardware stays, and how fairly work is shared, scheduling is one of the most heavily analysed topics in operating systems [1, 2, 3].

This article studies one such rule: Longest Job First (LJF). LJF always dispatches the ready process with the largest CPU burst time. It is the mirror image of Shortest Job First (SJF), and for that reason it is a valuable teaching device: contrasting the two exposes, with exact arithmetic, why the choice of selection key controls waiting time, fairness and starvation. Outside the classroom the same longest-first idea reappears in scheduling theory as the Longest Processing Time (LPT) rule for balancing work across identical machines [6].

The article is organised in fourteen chapters. Chapters 1–4 build the conceptual and procedural foundations; Chapters 5–7 develop five fully solved numerical examples, the mathematical analysis and the supporting figures; Chapters 8–9 weigh strengths, weaknesses and alternatives; Chapters 10–11 give working implementations and discuss applications; Chapters 12–14 cover improvements, conclusions and references. All numerical results were generated by one reference simulator and independently re-checked against three program implementations (Python, C++ and Java), so tables, Gantt charts and code output agree exactly.

Chapter 1: Fundamental Concepts

This chapter fixes the vocabulary used throughout the article: what an operating system does, what a process is, how it moves between states, and what a CPU scheduler must optimise.

1.1 Introduction to Operating Systems

An operating system (OS) is system software that manages hardware resources and provides services to application programs. Its principal duties are process management, memory management, file and storage management, device management and protection. Among these, process management is where CPU scheduling lives: the OS multiplexes a small number of processors among a much larger number of competing computations so that each appears to make progress [1].

1.2 Definition of a Process

A process is a program in execution. It comprises the program code, the current activity (program counter and registers), a stack, a data section and a heap. A program is a passive entity stored on disk, whereas a process is an active entity with a state. The OS keeps a Process Control Block (PCB) for each process, which stores its identifier, state, register contents, scheduling information and accounting data [1].

1.3 Process Life Cycle

During its lifetime a process moves among five states: New (being created), Ready (waiting for a CPU), Running (instructions are executing), Waiting (blocked for an I/O completion or event) and Terminated (finished). A scheduler is concerned with exactly one transition: Ready → Running, called dispatching. Figure 1 shows the states and the transitions, including the extra Running → Ready edge that exists only under preemptive policies.

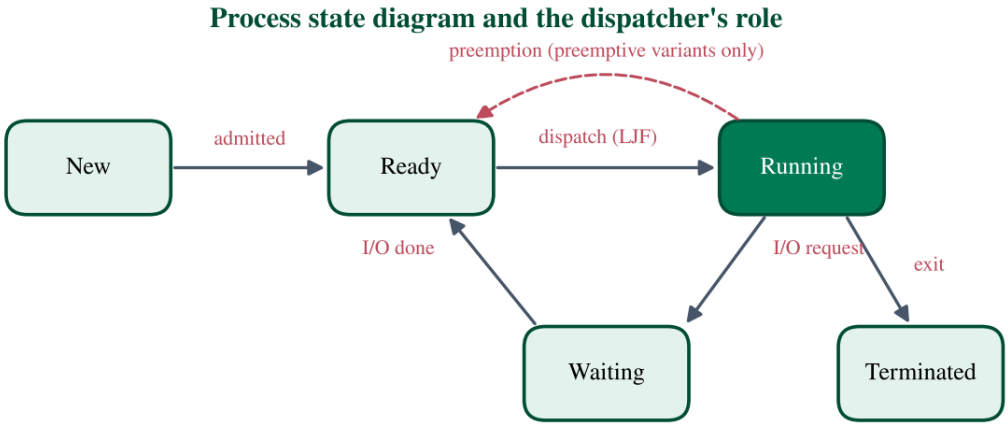


Figure 1: Process state diagram; the dispatch edge Ready → Running is where LJF makes its decision.

1.4 Definition of CPU Scheduling

CPU scheduling is the activity of selecting, from the set of processes in the ready queue, the one to which the CPU is allocated. Processes alternate between a CPU burst, a period of computation, and an I/O burst, a period of waiting for devices. The length of the next CPU burst is the Burst Time (BT) used by LJF. The short-term scheduler selects the process; the dispatcher then performs the context switch, which saves the state of the outgoing process and restores that of the incoming one [1, 3].

1.5 Importance of CPU Scheduling

The same set of processes can finish with very different waiting times depending only on the order of execution. Section 6.7 proves that for a batch of simultaneously arriving jobs the ordering alone can change the average waiting time by a factor of more than 1.7 (Example 1:

16.2 versus 9.4 time units). Scheduling therefore affects user-perceived responsiveness, hardware utilisation, throughput and the fairness of resource sharing.

1.6 Design Goals and Evaluation Criteria

Schedulers are evaluated against measurable criteria, which are formalised in Chapter 6. Their typical optimisation directions are given below.

- CPU utilisation: keep the CPU as busy as possible (maximise).
- Throughput: number of processes completed per unit time (maximise).
- Turnaround time: from arrival to completion (minimise).
- Waiting time: total time spent in the ready queue (minimise).
- Response time: from arrival to first CPU allocation (minimise).
- Fairness and absence of starvation: every process eventually runs.

1.7 Position of LJF among Scheduling Algorithms

Scheduling algorithms can be classified along two axes: whether they can take the CPU away from a running process (preemptive versus non-preemptive) and what key they use to rank ready processes (arrival order, burst length, priority, elapsed time). FCFS ranks by arrival; SJF and SRTF rank by short length; Round Robin ranks by rotation; Priority Scheduling ranks by an externally assigned number. LJF ranks by long length, so it occupies the corner opposite SJF. It is a theoretical and pedagogical policy rather than a mainstream general-purpose one (see Chapter 11).

Table 1: Basic concepts used in this article

Concept	Definition	Role in scheduling
Process	Program in execution with its own state	Unit that competes for the CPU
PCB	Kernel record describing a process	Stores state and scheduling data
CPU burst	Interval of continuous computation	Its length (BT) is LJF's selection key
I/O burst	Interval spent waiting for devices	Moves process to the waiting state
Ready queue	Set of runnable processes	Candidates for the next dispatch
Dispatcher	Performs the context switch	Gives the CPU to the chosen process
Context switch	Save/restore of CPU state	Pure overhead; frequent under preemption
Preemption	Forced removal of a running process	Distinguishes LJF from LRTF

Chapter summary

A scheduler's only task is the Ready → Running decision. LJF makes that decision by maximum burst time and is best understood as the opposite of SJF.

Chapter 2: The LJF Algorithm

This chapter defines Longest Job First precisely and examines the information it consumes, the queue it manages, and how it behaves under different arrival patterns.

2.1 Precise Definition

Longest Job First is a CPU-scheduling policy that, whenever the CPU becomes free, selects from the ready queue the process with the largest burst time. In its classical (non-preemptive) form the selected process runs to completion. Formally, if *R(t)* is the set of processes that have arrived by time *t* and not yet executed, the next process is *p^* = argmax_{p in R(t)} BT(p)*.

2.2 History and Context

There is no single canonical paper that introduces LJF as an operating-system algorithm; it appears mainly as the natural opposite of SJF in teaching material. Its theoretical ancestor is the LPT rule in machine-scheduling theory, whose approximation guarantee for minimising the finishing time on identical parallel machines was established by Graham [6, 7]. Optimality of the opposite rule, shortest-first, for the mean completion time on a single machine goes back to Smith [5]. Understanding LJF therefore means understanding which objective a longest-first rule serves (balanced finishing times on several machines) and which it harms (mean waiting time on one machine).

2.3 Core Idea and Logic

The rule gives priority to the largest job so that big computations start early. On one CPU this does not shorten the schedule, because the total busy time is the sum of all bursts whatever the order; it only redistributes waiting among processes, loading the cost onto the short ones. On several processors, however, starting long jobs first prevents a long job from being left alone at the end, which is exactly why LPT balances load well.

2.4 How the Next Process Is Selected

At every decision instant the scheduler (1) collects all unfinished processes with $AT <= t$, (2) picks the maximum BT, and (3) breaks ties deterministically. The tie-breaking rule used throughout this article is: earlier arrival first, then smaller process number. Making the rule explicit matters because Example 3 contains several equal bursts and different rules give different, equally valid schedules.

2.5 Entry of Processes into the Ready Queue

A process enters the ready queue at its arrival time (AT). A process that has not yet arrived is invisible to the scheduler, even if its burst is the largest in the whole workload. This property,

that LJF can only choose among what is currently present, is the source of the convoy effects discussed in Section 2.9.

2.6 Managing the Process Queue

Conceptually the ready queue is ordered by decreasing BT. Implementations may keep an unsorted list and scan it, keep a sorted list, or use a max-heap keyed by BT; these choices change the running time (Section 6.9) but never the schedule. If no process is ready, the CPU idles until the next arrival.

2.7 The Role of Burst Time

BT is the only quantity compared. The algorithm needs it before execution starts, but a real OS does not know the future. It must predict BT, commonly by exponential averaging of previous bursts, *tau_(n+1) = alpha t_n + (1 − alpha) tau_n* with *0 <= alpha <= 1* [1]. LJF therefore inherits the prediction problem of SJF, and prediction errors can reverse the intended order.

2.8 Arrival Time versus Execution Time

Arrival time says when a process becomes eligible; burst time says how long it needs the CPU. They are independent attributes. Arrival time only gates eligibility, while burst time decides the ranking. A process with a huge burst that arrives late cannot overtake a running job in the non-preemptive version.

2.9 Behaviour under Different Conditions

- All processes arrive together: LJF reduces to sorting by descending BT (Example 1).
- Staggered arrivals: early short jobs may be started simply because nothing longer is present; later arrivals reorder the queue (Example 2).
- Equal bursts: the result is determined by the tie-break rule and, with a first-come tie-break, coincides with FCFS among the tied jobs (Example 3).
- Idle gaps: if the queue empties, the CPU idles and utilisation drops below 100%.
- Continuous long arrivals: short jobs can wait indefinitely, i.e. starvation (Section 8.2).
- One very long job: it is served first when present, and every other process waits behind it.

> **Chapter summary**
>
> LJF selects argmax BT among arrived, unfinished processes, breaks ties by earlier arrival, and needs BT in advance. Its behaviour ranges from a simple sort (common arrival) to starvation (continuous long arrivals).

Chapter 3: Types of LJF

LJF exists in a non-preemptive form and in preemptive relatives. The terminology is easily confused, so this chapter defines each variant carefully.

3.1 Non-Preemptive LJF

Once a process receives the CPU it keeps it until it terminates. Decisions are taken only at completion instants (and after idle periods). This is the algorithm referred to simply as LJF in the rest of the article, and the one implemented in Chapter 10.

3.2 Preemptive LJF

A preemptive policy may stop the running process before it finishes. For a longest-first rule there are two distinct ways to define this, and the literature uses the name loosely:

- Static-key preemptive LJF (our name: LJF-PA, preemptive on arrival). The ranking key stays equal to the original burst time. A new arrival preempts the running process only if its burst is larger than the running process's original burst.
- Dynamic-key preemptive LJF, universally called Longest Remaining Time First (LRTF). The key is the remaining burst time, which decreases while a process runs. The running process can therefore lose the CPU even without a new arrival.

3.3 The Concept of Preemption

Preemption is the involuntary removal of the CPU from a running process, triggered by a timer or by the arrival of a better-ranked process. It makes a scheduler more reactive but adds context-switch overhead and requires saving the state of every interrupted process [1].

3.4 Preemptive versus Non-Preemptive

Table 2: Comparison of non-preemptive LJF, LJF-PA and LRTF

Aspect	LJF (non-preemptive)	LJF-PA	LRTF
Ranking key	Original BT	Original BT	Remaining BT
Decision instants	Completions	Completions and arrivals	Every time unit
Can interrupt a process	No	Only on arrival of a larger job	Yes, including without arrivals
Context switches	Fewest	Few	Many
Tends to equalise finish times	No	No	Yes (processes converge to the same remaining time)
Implementation	Simple	Moderate	Needs timer or event simulation

3.5 When the Running Process Changes

In non-preemptive LJF the running process changes only when it terminates. In LJF-PA it changes when a process arrives whose original BT exceeds that of the running process. In LRTF it changes whenever some ready process has a strictly larger remaining time than the running one, which can occur at an arrival or when the running process has consumed enough time that its remaining burst drops below another's.

3.6 Advantages and Drawbacks of Each Version

Non-preemptive LJF is simple and has minimal overhead, but a late-arriving long job must wait for the current job. LJF-PA lets a long job start immediately, but it can leave a partially executed job unfinished for a long time. LRTF reacts best to the current workload and tends to make all jobs finish near the end, which suits makespan-driven batch work, but its constant switching creates the highest overhead and, in the extreme, processor sharing among tied jobs. All variants share LJF's weakness for short jobs.

3.7 Independent Examples for Each Version

Take the workload of Example 5 (Section 5.5): P1(AT 0, BT 4), P2(1, 7), P3(2, 3), P4(4, 6). Non-preemptive LJF yields P1 [0–4] | P2 [4–11] | P4 [11–17] | P3 [17–20]. LJF-PA runs P1 only until P2 arrives (its BT 7 exceeds 4), giving P1 [0–1] | P2 [1–8] | P4 [8–14] | P1 [14–17] | P3 [17–20]. LRTF produces the fragmented schedule P1 [0–1] | P2 [1–4] | P4 [4–7] | P2 [7–9] | P1 [9–10] | P3 [10–11] | P4 [11–13] | P1 [13–14] | P2 [14–15] | P3 [15–17] | P1 [17–18] | P2 [18–19] | P4 [19–20].

Table 3: Results of the three variants on the Example 5 workload

Variant	AWT	ATAT	ART	Preemptions
LJF (non-preemptive)	6.25	11.25	6.25	0
LJF-PA	8.00	13.00	4.75	1
LRTF	11.75	16.75	2.00	9

3.8 Related Terminology: LJF versus LRTF

> **LJF and LRTF are not the same algorithm**
>
> LJF ranks by the total burst time and, in its classical form, never interrupts a running process. LRTF ranks by the remaining burst time and may interrupt at any instant. They coincide only for special inputs, for example when a single process is present. In general they produce different schedules and different metrics (Table 3 and Example 5), so this article never treats one as a synonym of the other.

Chapter 4: Operation and Execution Steps

This chapter turns the definition into a step-by-step procedure and a flowchart.

4.1 Step-by-Step Procedure

1. Receive the process data: for every process, an identifier, arrival time and burst time.
2. Record the arrival time (AT) of each process.
3. Record the burst time (BT) of each process.
4. Form the ready queue: at the current time t, include every unfinished process with $AT <= t$.
5. Compare the burst times of the processes in the queue.
6. Select the process with the largest burst time (tie: earlier AT, then smaller index).
7. Allocate the CPU to it; its start time is $ST = t$.
8. Manage its completion: $CT = ST + BT$ and $t = CT$.
9. Update the queue: remove the finished process and admit processes that arrived during its run.
10. Repeat steps 4–9 until every process has completed; if the queue is empty, advance t to the next arrival.

4.2 Flowchart

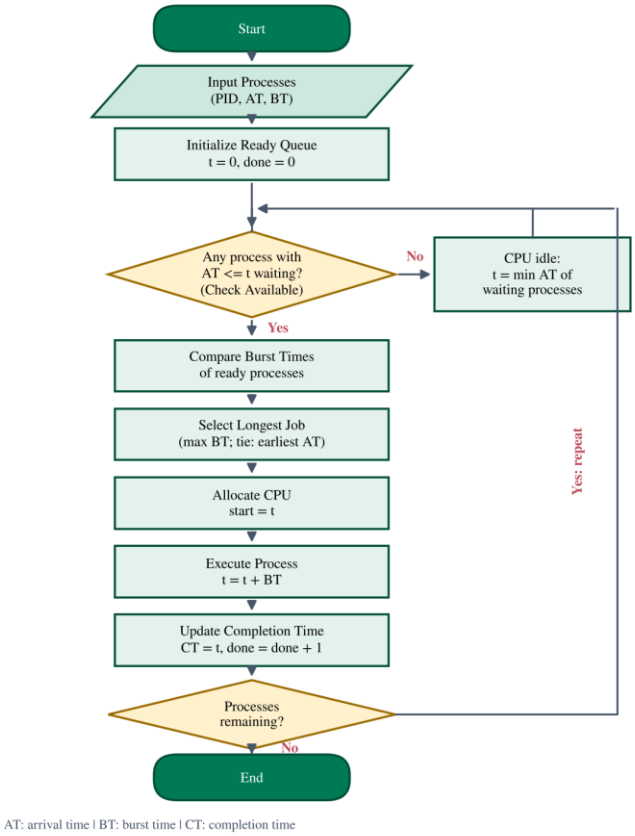


Figure 2: Flowchart of non-preemptive LJF, including the idle-CPU branch.

4.3 Pseudo-code

Algorithm 1: Non-preemptive LJF

```
t = 0; done = 0
while done < n:
    ready = {p : p.AT <= t and not p.finished}
    if ready is empty:
        t = min(p.AT for p unfinished)   # CPU idle
        continue
    p* = argmax(ready, key = BT)         # tie: earlier AT, smaller index
    p*.ST = t;  t = t + p*.BT;  p*.CT = t
    p*.finished = true;  done = done + 1
```

Chapter summary

The procedure alternates between forming the ready queue, choosing the maximum BT and advancing the clock; idle gaps are handled by jumping to the next arrival.

Chapter 5: Numerical Examples and Step-by-Step Solutions

Five examples cover the principal situations. Throughout, ST is the start time and CT the completion time of the (single) execution of each process; TAT = CT − AT, WT = TAT − BT and RT = ST − AT (formalised in Chapter 6). All tables are produced by the reference simulator and re-verified by hand below.

5.1 Example 1: All Processes Arrive at Time 0

Table 4: Process specifications of Example 1

PID	AT	BT
P1	0	6
P2	0	8
P3	0	3
P4	0	10
P5	0	5

Because all processes are present at $t = 0$, LJF simply sorts them by decreasing burst time. The selection order is P4, P2, P1, P5, P3.

- At $t = 0$ the ready queue is {P4 (10), P2 (8), P1 (6), P5 (5), P3 (3)}. The largest burst time is 10 (process P4), so P4 runs during [0, 10).
- At $t = 10$ the ready queue is {P2 (8), P1 (6), P5 (5), P3 (3)}. The largest burst time is 8 (process P2), so P2 runs during [10, 18).
- At $t = 18$ the ready queue is {P1 (6), P5 (5), P3 (3)}. The largest burst time is 6 (process P1), so P1 runs during [18, 24).
- At $t = 24$ the ready queue is {P5 (5), P3 (3)}. The largest burst time is 5 (process P5), so P5 runs during [24, 29).

- At $t = 29$ the ready queue is {P3 (3)}. The largest burst time is 3 (process P3), so P3 runs during [29, 32).

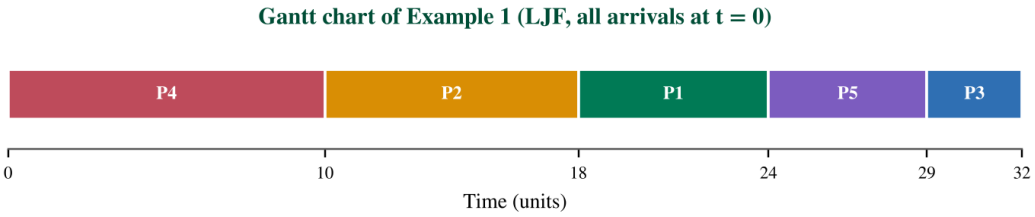


Figure 3: Gantt chart of Example 1.

Table 5: Results of Example 1 under LJF

PID	AT	BT	ST	CT	TAT	WT	RT
P1	0	6	18	24	24	18	18
P2	0	8	10	18	18	10	10
P3	0	3	29	32	32	29	29
P4	0	10	0	10	10	0	0
P5	0	5	24	29	29	24	24
Average					22.60	16.20	16.20

Analysis. P4 waits 0 units but P3, the shortest job, waits 29 units, more than three times its own burst. Average waiting time is 81/5 = 16.20; average turnaround time is 113/5 = 22.60. CPU utilisation is 100% and throughput is 5/32 = 0.156 processes per time unit. Because all arrivals are simultaneous, WT equals RT for every process.

5.2 Example 2: Different Arrival Times

Table 6: Process specifications of Example 2

PID	AT	BT
P1	0	4
P2	1	3
P3	2	6
P4	3	2
P5	5	5

Here the choice at each completion is limited to processes that have already arrived. At $t = 0$ only P1 is present, so it starts even though it is not the longest process overall.

- At $t = 0$ the ready queue is {P1 (4)}. The largest burst time is 4 (process P1), so P1 runs during [0, 4).
- At $t = 4$ the ready queue is {P3 (6), P2 (3), P4 (2)}. The largest burst time is 6 (process P3), so P3 runs during [4, 10).

- At $t = 10$ the ready queue is {P5 (5), P2 (3), P4 (2)}. The largest burst time is 5 (process P5), so P5 runs during [10, 15).
- At $t = 15$ the ready queue is {P2 (3), P4 (2)}. The largest burst time is 3 (process P2), so P2 runs during [15, 18).
- At $t = 18$ the ready queue is {P4 (2)}. The largest burst time is 2 (process P4), so P4 runs during [18, 20).

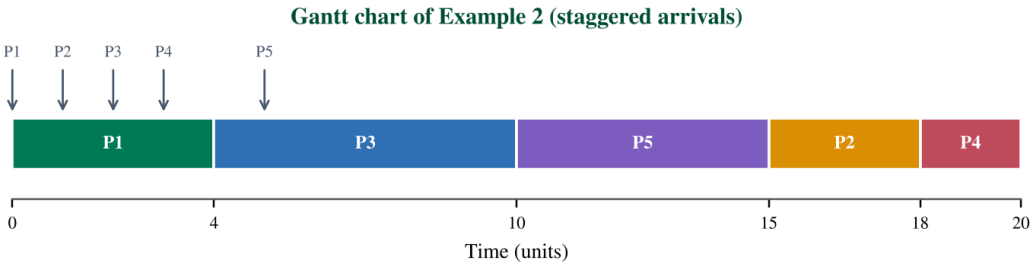


Figure 4: Gantt chart of Example 2 (arrows mark arrivals).

Table 7: Completion time calculation for Example 2

PID	ST	BT	CT = ST + BT
P1	0	4	0 + 4 = 4
P2	15	3	15 + 3 = 18
P3	4	6	4 + 6 = 10
P4	18	2	18 + 2 = 20
P5	10	5	10 + 5 = 15

Table 8: Turnaround time calculation for Example 2

PID	CT	AT	TAT = CT − AT
P1	4	0	4 − 0 = 4
P2	18	1	18 − 1 = 17
P3	10	2	10 − 2 = 8
P4	20	3	20 − 3 = 17
P5	15	5	15 − 5 = 10

Table 9: Waiting time calculation for Example 2

PID	TAT	BT	WT = TAT − BT
P1	4	4	4 − 4 = 0
P2	17	3	17 − 3 = 14
P3	8	6	8 − 6 = 2
P4	17	2	17 − 2 = 15
P5	10	5	10 − 5 = 5

Table 10: Summary of results of Example 2

PID	AT	BT	ST	CT	TAT	WT	RT
P1	0	4	0	4	4	0	0
P2	1	3	15	18	17	14	14
P3	2	6	4	10	8	2	2
P4	3	2	18	20	17	15	15
P5	5	5	10	15	10	5	5
Average					11.20	7.20	7.20

Analysis. The two shortest jobs, P4 (BT 2) and P2 (BT 3), finish last and wait 15 and 14 units respectively, a typical LJF pattern. Average waiting time is 36/5 = 7.20. For comparison, FCFS gives 5.60 and SJF gives 4.40 on the same workload.

5.3 Example 3: Equal Burst Times

Table 11: Process specifications of Example 3

PID	AT	BT
P1	0	5
P2	1	8
P3	2	8
P4	3	3
P5	4	8
P6	6	3

Three processes (P2, P3, P5) share the largest burst, 8, and two (P4, P6) share the smallest, 3. With the tie-break rule 'earlier arrival first', ties are resolved in arrival order.

- At $t = 0$ the ready queue is {P1 (5)}. The largest burst time is 5 (process P1), so P1 runs during [0, 5).
- At $t = 5$ the ready queue is {P2 (8), P3 (8), P5 (8), P4 (3)}. The largest burst time is 8 (process P2), so P2 runs during [5, 13).
- At $t = 13$ the ready queue is {P3 (8), P5 (8), P4 (3), P6 (3)}. The largest burst time is 8 (process P3), so P3 runs during [13, 21).
- At $t = 21$ the ready queue is {P5 (8), P4 (3), P6 (3)}. The largest burst time is 8 (process P5), so P5 runs during [21, 29).
- At $t = 29$ the ready queue is {P4 (3), P6 (3)}. The largest burst time is 3 (process P4), so P4 runs during [29, 32).
- At $t = 32$ the ready queue is {P6 (3)}. The largest burst time is 3 (process P6), so P6 runs during [32, 35).

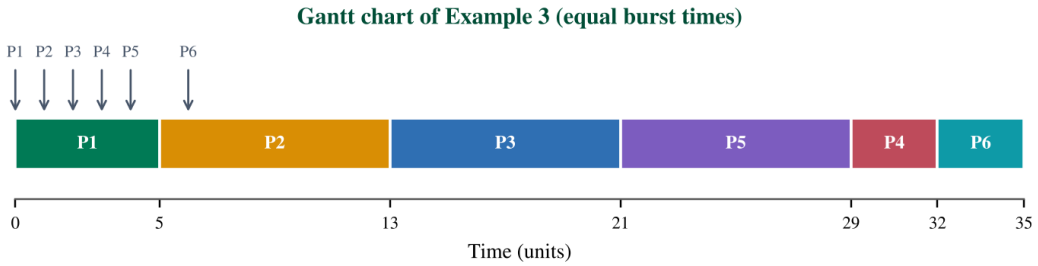


Figure 5: Gantt chart of Example 3 (equal bursts).

Table 12: Results of Example 3 under LJF

PID	AT	BT	ST	CT	TAT	WT	RT
P1	0	5	0	5	5	0	0
P2	1	8	5	13	12	4	4
P3	2	8	13	21	19	11	11
P4	3	3	29	32	29	26	26
P5	4	8	21	29	25	17	17
P6	6	3	32	35	29	26	26
Average					19.83	14.00	14.00

Analysis. Within each tied group the schedule is FCFS, so P2, P3, P5 run in arrival order and so do P4, P6. Average waiting time is 84/6 = 14.00. A different tie-break, such as the smallest process number, would give the same order here because arrival order and index coincide, but in general the tie-break rule can change individual waiting times and must therefore be stated explicitly.

5.4 Example 4: Ten Processes and Ready-Queue Analysis

Table 13: Process specifications of Example 4

PID	AT	BT
P1	0	7
P2	1	4
P3	2	9
P4	3	3
P5	4	6
P6	6	8
P7	8	2
P8	9	5
P9	12	10
P10	14	1

Table 14: Ready queue at each decision instant in Example 4 (ordered by decreasing BT)

t	Ready queue (BT)	Selected
0	{P1 (7)}	P1
7	{P3 (9), P6 (8), P5 (6), P2 (4), P4 (3)}	P3
16	{P9 (10), P6 (8), P5 (6), P8 (5), P2 (4), P4 (3), P7 (2), P10 (1)}	P9
26	{P6 (8), P5 (6), P8 (5), P2 (4), P4 (3), P7 (2), P10 (1)}	P6
34	{P5 (6), P8 (5), P2 (4), P4 (3), P7 (2), P10 (1)}	P5
40	{P8 (5), P2 (4), P4 (3), P7 (2), P10 (1)}	P8
45	{P2 (4), P4 (3), P7 (2), P10 (1)}	P2
49	{P4 (3), P7 (2), P10 (1)}	P4
52	{P7 (2), P10 (1)}	P7
54	{P10 (1)}	P10

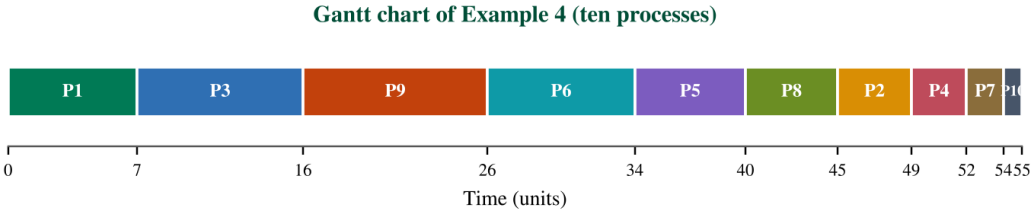


Figure 6: Gantt chart of Example 4.

Table 15: Results of Example 4 under LJF

PID	AT	BT	ST	CT	TAT	WT	RT
P1	0	7	0	7	7	0	0
P2	1	4	45	49	48	44	44
P3	2	9	7	16	14	5	5
P4	3	3	49	52	49	46	46
P5	4	6	34	40	36	30	30
P6	6	8	26	34	28	20	20
P7	8	2	52	54	46	44	44
P8	9	5	40	45	36	31	31
P9	12	10	16	26	14	4	4
P10	14	1	54	55	41	40	40
Average					31.90	26.40	26.40

Analysis. The ready queue grows from 1 process at $t = 0$ to a maximum of 8 at $t = 16$, then drains one process per decision. The three shortest jobs, P10 (BT 1), P7 (BT 2) and P4 (BT 3), complete last and wait 40, 44 and 46 units, which is 40, 22 and 15 times their own burst. By Little's law the mean number of waiting processes equals the total waiting time divided by the elapsed time: 264/55 = 4.80. Average waiting time is 26.40 and average turnaround time is 31.90.

5.5 Example 5: Preemptive Version (LRTF) versus Non-Preemptive LJF

Table 16: Process specifications of Example 5

PID	AT	BT
P1	0	4
P2	1	7
P3	2	3
P4	4	6

Non-preemptive LJF runs P1 for 4 units because no longer process is present when it starts. When P2 (BT 7) arrives at $t = 1$ it must wait until $t = 4$. LRTF compares remaining times at every unit: at $t = 1$ P2 has remaining 7 versus P1's remaining 3, so P2 preempts P1 and runs during [1, 4), its remaining time falling from 7 to 4. When P4 arrives at $t = 4$ with remaining 6 against P2's remaining 4, P4 preempts P2, and so on; whenever the running process ties for the longest remaining time it is kept, which avoids needless switches.

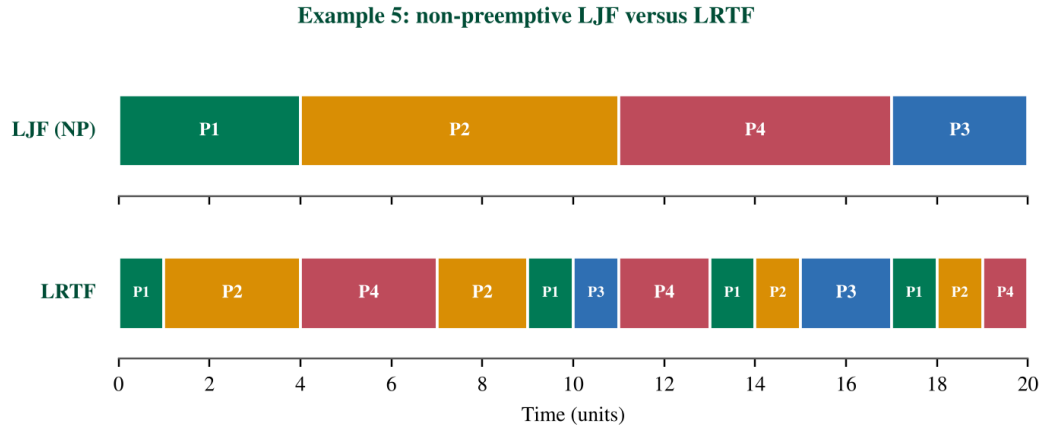


Figure 7: Example 5: non-preemptive LJF (top) versus LRTF (bottom).

Table 17: Example 5 results: LJF (non-preemptive)

PID	AT	BT	ST	CT	TAT	WT	RT
P1	0	4	0	4	4	0	0
P2	1	7	4	11	10	3	3
P3	2	3	17	20	18	15	15
P4	4	6	11	17	13	7	7
Average					11.25	6.25	6.25

Table 18: Example 5 results: LRTF (preemptive)

PID	AT	BT	ST	CT	TAT	WT	RT
P1	0	4	0	18	18	14	0
P2	1	7	1	19	18	11	0
P3	2	3	10	17	15	12	8
P4	4	6	4	20	16	10	0
Average					16.75	11.75	2.00

Analysis. Under LRTF all four processes finish between t = 17 and t = 20, whereas under LJF they finish at 4, 11, 17 and 20, so LRTF equalises finishing times, as theory predicts, at the price of 9 preemptions. Its average waiting time (11.75) is much higher than LJF's (6.25), while its average response time is lower (2.00 versus 6.25) because every process receives the CPU soon after arriving, except P3. In LRTF, ST is the first start time; CT is the last completion instant and WT = TAT − BT still holds even though the execution is fragmented. The makespan is 20 in both cases, since a single CPU has no idle time here.

Chapter summary

Five workloads show LJF as a descending sort (Example 1), a constrained choice (Example 2), a tie-break-dependent choice (Example 3), a queue that builds up and starves short jobs (Example 4), and the clear distinction from LRTF (Example 5).

Chapter 6: Formulas and Mathematical Analysis

All metrics are defined for a set of n processes. Let AT_i, BT_i, ST_i and CT_i denote arrival, burst, first start and completion times of process i.

6.1 Turnaround Time

$$\text{TAT} = \text{Completion Time} - \text{Arrival Time} \tag{1}$$

Turnaround time is the total time a process spends in the system, from submission to completion. It is the batch-processing view of latency. Example: in Example 2, P2 arrives at 1 and completes at 18, so TAT = 18 − 1 = 17. A value much larger than BT means the process spent most of its life waiting.

6.2 Waiting Time

$$\text{WT} = \text{Turnaround Time} - \text{Burst Time} \tag{2}$$

Waiting time is the part of the turnaround spent in the ready queue. For a non-preemptive scheduler it also equals the first start time minus the arrival time. Example: P4 in Example 2 has TAT = 17 and BT = 2, so WT = 15. It is the metric on which LJF performs worst.

6.3 Average Waiting Time

$$\text{AWT} = (\text{sum of WT_i}) / n \tag{3}$$

AWT summarises the system-wide waiting cost and is the criterion minimised by SJF. Example: in Example 1, AWT = (18 + 10 + 29 + 0 + 24)/5 = 81/5 = 16.2. Values should be compared only for the same workload.

6.4 Average Turnaround Time

$$\text{ATAT} = (\text{sum of TAT_i}) / n \tag{4}$$

The average turnaround time is the mean time to completion. Example: Example 2 gives ATAT = (4 + 17 + 8 + 17 + 10)/5 = 56/5 = 11.2. Since TAT_i = WT_i + BT_i, ATAT = AWT + mean(BT) always holds (11.2 = 7.2 + 4.0).

6.5 Response Time

$$\text{RT} = \text{First CPU Start Time} - \text{Arrival Time} \tag{5}$$

Response time measures how soon a process first gets the CPU, which matters for interactive use. For non-preemptive algorithms RT = WT. Example: P5 in Example 2 arrives at 5 and first starts at 10, so RT = 5. In LRTF, RT can be much smaller than WT (Example 5).

6.6 CPU Utilisation and Throughput

$$\text{CPU Utilisation} = (\text{CPU Busy Time} / \text{Total Elapsed Time}) \times 100 \tag{6}$$

$$\text{Throughput} = \text{Number of Completed Processes} / \text{Total Elapsed Time} \tag{7}$$

Utilisation measures how much of the interval is spent executing processes; throughput measures completion rate. Example: take P1(AT 0, BT 3) and P2(AT 10, BT 4). LJF runs P1 over [0, 3) and P2 over [10, 14); the CPU idles for 7 units, so utilisation = 7/14 × 100 = 50% and throughput = 2/14 = 0.143 per unit. In Example 1 there are no gaps, so utilisation is 100% and throughput is 5/32 = 0.156. Both metrics are independent of the order of execution when no idle time arises, so they do not distinguish LJF from SJF.

Table 19: Summary of formulas

Metric	Formula	Goal	Example 2 value
TAT	CT − AT	min	P2: 17
WT	TAT − BT	min	P4: 15
AWT	ΣWT / n	min	7.2
ATAT	ΣTAT / n	min	11.2
RT	ST − AT	min	P5: 5
Utilisation	Busy / Elapsed × 100	max	100%
Throughput	Completed / Elapsed	max	0.25

6.7 Optimality Analysis: Why LJF Maximises the Average Waiting Time

Consider n jobs that are all available at time 0 and are run in the order σ(1), …, σ(n) with burst times *b_1, ..., b_n* in that order. The job in position k waits for all earlier jobs, so each burst *b_k* is counted once for every later job:

$$W(\sigma) = \Sigma_{k=1..n} (n - k) \cdot b_k \tag{8}$$

Swap two adjacent jobs x (position k) and y (position k + 1). The change in W is (y − x), because the earlier position carries the larger multiplier. If x > y, placing x first makes W larger. Repeating such swaps shows that the descending order (LJF) maximises W and the ascending order (SJF) minimises it, a special case of the rearrangement inequality [5]. For Example 1 an exhaustive search over all 5! = 120 orderings confirms it: the total waiting time ranges from 47 (SJF) to 81 (LJF), with FCFS at 64 (Figure 8). Hence LJF is the worst possible non-preemptive order with respect to average waiting time when arrivals coincide.

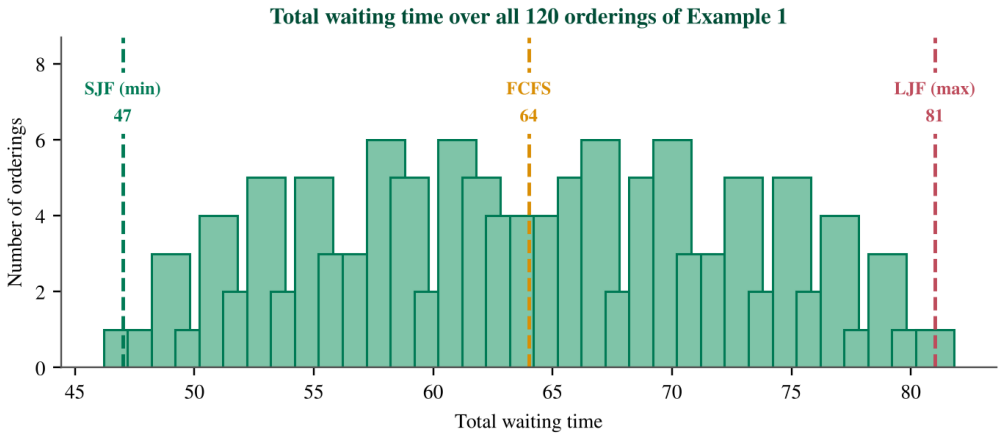


Figure 8: Total waiting time of all 120 orderings of the Example 1 workload.

6.8 Where Longest-First Is Good: Makespan on Parallel Machines

On m identical processors the objective is usually the makespan, the time at which the last job ends. Assigning jobs in non-increasing order to the least-loaded machine is the LPT rule;

Graham proved that its makespan is at most (4/3 − 1/(3m)) times the optimal one [7]. This guarantee explains the practical interest in longest-first ordering and shows that its poor waiting-time behaviour on one CPU is the price of optimising a different objective.

6.9 Time and Space Complexity

Let n be the number of processes. A naive implementation scans the whole list at each of the n decisions: O(n) per decision and O(n²) in total. Keeping a list sorted by BT makes selection O(1), but inserting an arriving process costs O(n), so the total remains O(n²) in the worst case. A binary max-heap keyed by BT supports insertion and extraction in O(log n); with the processes first sorted by arrival time in O(n log n), the whole schedule is computed in O(n log n) [4]. Space is O(n) in all variants. Figure 9 contrasts the growth of linear scanning and heap-based selection.

Table 20: Time and space complexity of LJF implementations

Implementation	Selection	Insertion	Total time	Space
Unsorted list, linear scan	O(n)	O(1)	O(n²)	O(n)
List kept sorted by BT	O(1)	O(n)	O(n²)	O(n)
Priority queue as max-heap	O(log n)	O(log n)	O(n log n)	O(n)
LRTF, unit-time simulation	O(log n) per unit	O(log n)	O(T log n), T = sum of BT	O(n)

Assumptions: n processes, comparison-based selection, arrival times sorted once in O(n log n). T denotes the total burst time, so the LRTF bound is pseudo-polynomial.

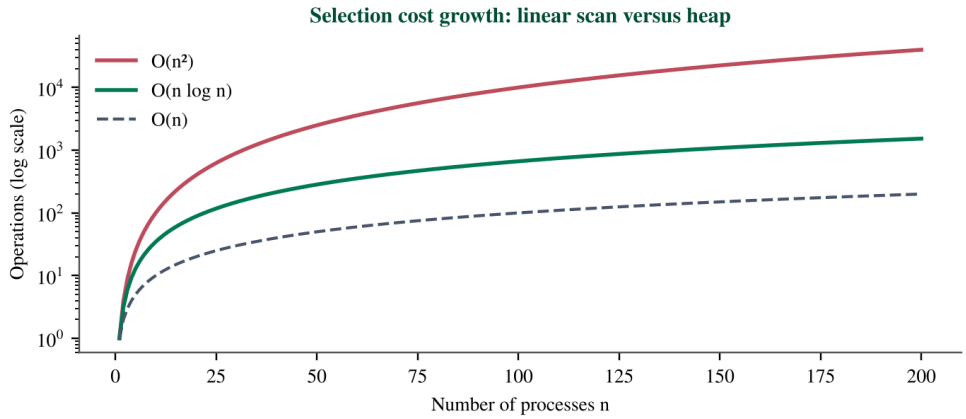


Figure 9: Growth of linear-scan (n²) versus heap-based (n log n) total selection cost.

Chapter summary

The metrics are simple linear quantities; the key theoretical result is that LJF maximises, and SJF minimises, the average waiting time for simultaneous arrivals, while longest-first is a sound heuristic for makespan on parallel machines.

Chapter 7: Diagrams and Educational Figures

In addition to the flowchart and the Gantt charts of the examples, this chapter collects comparative figures. Table 21 lists every figure of the article with its purpose.

Table 21: Catalogue of figures

Figure	Content	Teaching purpose
1	Process state diagram	Locate the dispatch decision
2	LJF flowchart	Show the algorithm
3–6	Gantt charts, Examples 1–4	Visualise schedules
7	LJF versus LRTF Gantt	Show effect of preemption
8–9	Permutation histogram, complexity growth	Support the analysis
10	Execution-order comparison	Contrast FCFS, SJF, LJF
11	Ready-queue length (Example 4)	Show queue build-up
12–13	Waiting / turnaround per process	Compare policies per job
14–15	LJF vs FCFS, LJF vs SJF	Quantify the penalty

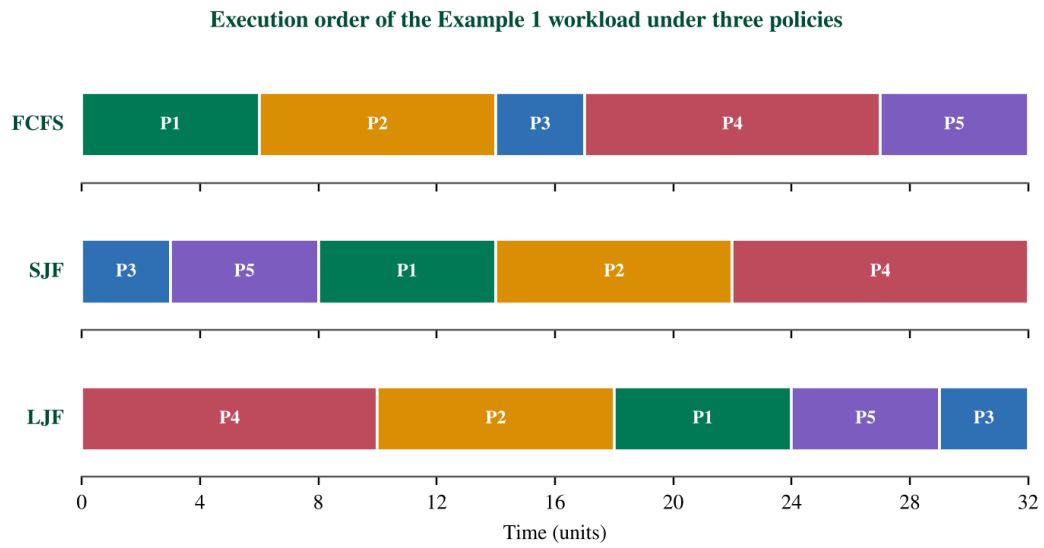


Figure 10: Execution order of the Example 1 workload under FCFS, SJF and LJF.

Figure 10 shows the same five processes in three orders. FCFS follows the numbering, SJF runs the shortest first and LJF reverses SJF exactly; the finishing time of the last process, 32, is identical, illustrating that order changes who waits, not how much work is done.

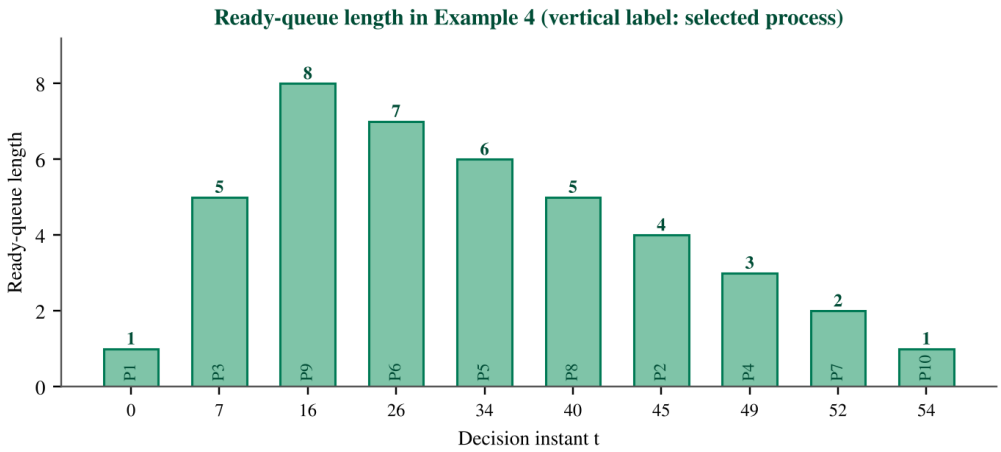


Figure 11: Ready-queue length at the decision instants of Example 4.

Figure 11 shows that the ready queue peaks at 8 processes at $t = 16$ and then shrinks by one at each decision; the long right-hand tail is populated by the short jobs that LJF postpones.

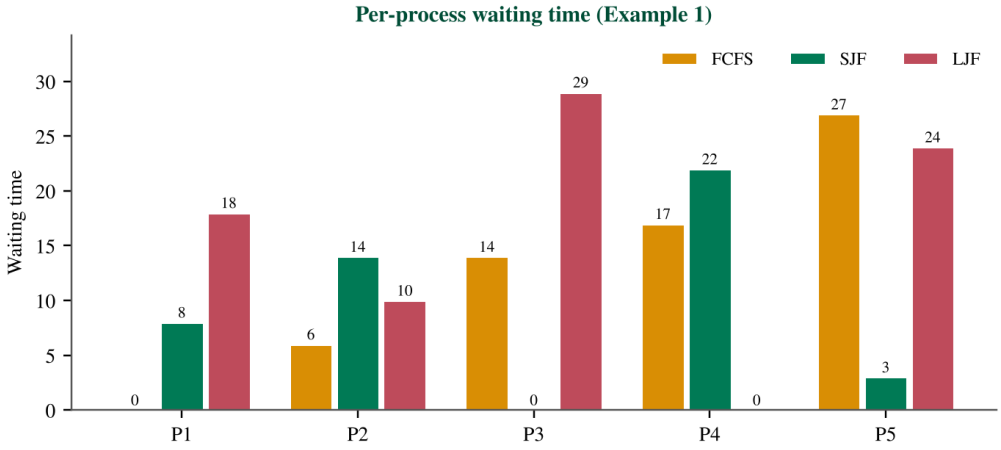


Figure 12: Waiting time of each process of Example 1 under FCFS, SJF and LJF.

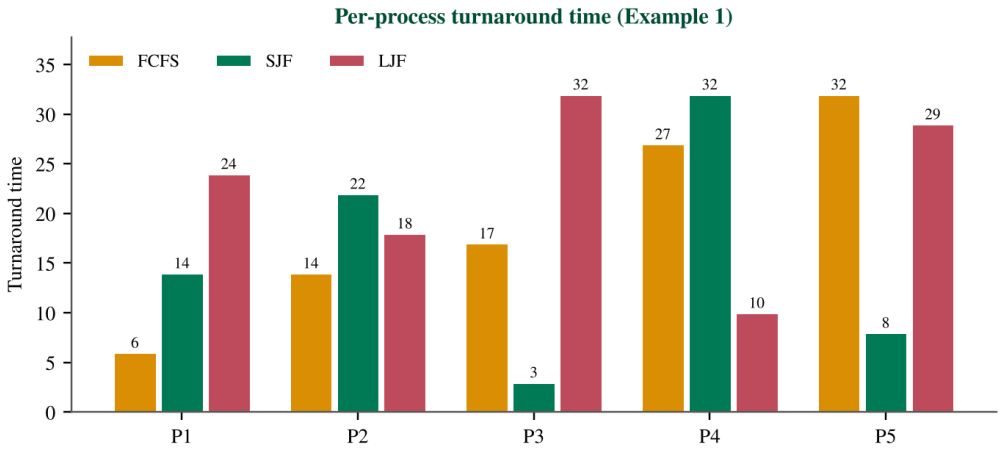


Figure 13: Turnaround time of each process of Example 1 under FCFS, SJF and LJF.

Figures 12 and 13 show that LJF gives the smallest waiting time to the longest process (P4: 0) and the largest to the shortest one (P3: 29), the reverse of SJF. The average waiting times are

12.80 (FCFS), 9.40 (SJF) and 16.20 (LJF); the corresponding average turnaround times are 19.20, 15.80 and 22.60.

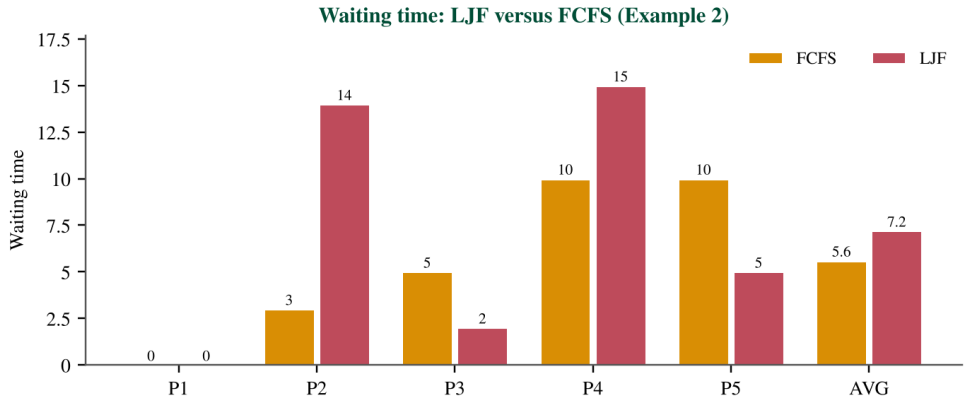


Figure 14: Waiting time of Example 2 under LJF and FCFS.

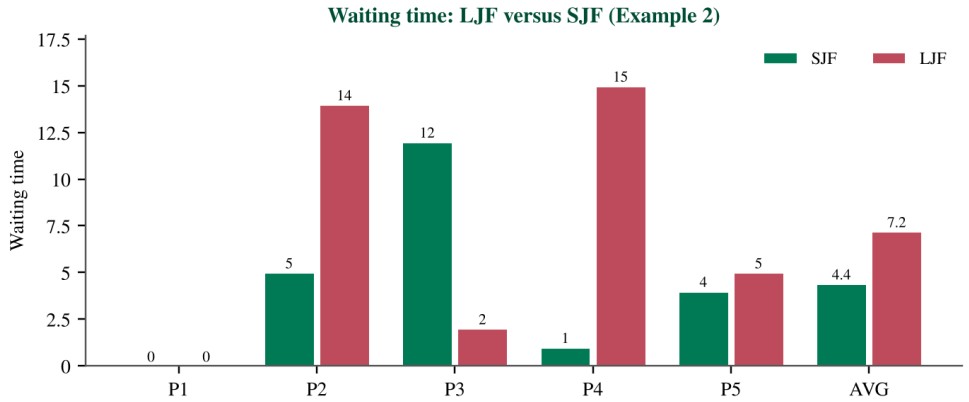


Figure 15: Waiting time of Example 2 under LJF and SJF.

For Example 2, LJF has average waiting time 7.20 versus 5.60 for FCFS and 4.40 for SJF, a penalty of 64% over SJF.

Chapter summary

The figures confirm that LJF leaves total work unchanged while redistributing waiting time toward short processes.

Chapter 8: Advantages and Disadvantages of LJF

8.1 Advantages

- Conceptual simplicity. One comparison (largest BT) defines the policy; it is easy to explain and to simulate.
- Easy basic implementation. A scan or sort suffices (Chapter 10), with no timer or preemption support.
- Early service for long processes. In Example 1 the largest job P4 starts at 0 and has TAT 10, the smallest of all.

- Analysability. The exchange argument of Section 6.7 gives exact results, making LJF a clean test case for proofs and simulators.
- Educational value. Comparing LJF with SJF isolates the effect of the ranking key while keeping everything else equal.

8.2 Disadvantages

8.2.1 Long waiting for short processes

The shortest jobs wait for all longer ones. In Example 4 process P10 (BT 1) waits 40 units; the ratio WT/BT = 40 is a measure of severe inconvenience.

8.2.2 Starvation of short jobs

Suppose a long job (BT 6) is running when a short job S (BT 2) arrives at t = 1, and a new long job arrives every time unit for five more units. LJF runs all long jobs first and S finishes at 38, after waiting 35 units, while SJF would make S wait only 5. If long jobs keep arriving, S waits without bound: this is starvation.

8.2.3 Unfavourable average waiting time

As proved in Section 6.7, LJF yields the maximum AWT among all non-preemptive orders when arrivals coincide; in Example 1 its AWT is 72% above SJF's.

8.2.4 Dependence on burst-time estimates

The policy needs BT before execution. Estimates from exponential averaging can be wrong, and because LJF amplifies the cost of misplacing a short job behind a long one, errors have a larger impact than for SJF.

8.2.5 Unsuitability for interactive systems

Interactive users expect a quick first response. Since LJF may delay any short request behind arbitrarily long jobs and is non-preemptive, response times are unbounded. Time-sharing systems use Round Robin or multilevel feedback queues instead [1, 3].

8.2.6 Practical limits

General-purpose operating systems do not know burst times in advance, serve mixed interactive and batch workloads, and must guarantee fairness, so LJF is rarely their primary policy.

Table 22: Advantages and disadvantages of LJF

Aspect	Advantage	Disadvantage
Simplicity	One-rule policy, simple code	Hidden need for BT knowledge
Long jobs	Start early, low TAT	Dominate the CPU
Short jobs	—	Long waits, starvation risk
Waiting time	—	Maximal AWT for common arrivals
Analysis	Exact, provable results	Results are unfavourable
Interactivity	—	Unbounded response time
Use in practice	Teaching, parallel makespan heuristics	Rare in general-purpose OS

Chapter summary

LJF's strengths are simplicity and analysability; its weaknesses (waiting time, starvation, estimation, interactivity) explain its limited practical use.

Chapter 9: Comparison with Other Algorithms

9.1 Qualitative Comparison

Table 23: Comprehensive comparison of scheduling algorithms

Algorithm	Selection rule	Preemptive	Pros	Cons	Complexity	Typical use
FCFS	Earliest arrival	No	Simple, fair order	Convoy effect	O(1) per decision	Batch, baseline
SJF	Smallest BT	No	Minimal AWT (common arrival)	Starvation, needs BT	O(log n) heap	Batch, theory
SRTF	Smallest remaining	Yes	Optimal AWT with arrivals	Many switches, starvation	O(log n) per event	Theory, benchmarks
Round Robin	Rotation, quantum q	Yes	Good response, no starvation	Higher AWT, overhead	O(1) per decision	Time-sharing
Priority	Highest priority	Either	Flexible policy	Starvation without aging	O(log n) heap	Real-time, general OS
LJF	Largest BT	No	Simple, favours long jobs	Maximal AWT, starvation	O(log n) heap	Teaching, makespan
LRTF	Largest remaining	Yes	Equal finish times	Heavy switching, high AWT	O(T log n)	Theory, makespan

9.2 Behaviour under Different Conditions

- Common arrival: SJF is optimal for AWT, LJF is worst, FCFS is in between.
- Staggered arrivals: SRTF is optimal; preemption lets short newcomers run at once.
- Highly variable burst lengths: FCFS and LJF suffer most because short jobs queue behind long ones.

- Interactive load: Round Robin and priority-with-aging dominate; LJF and SJF are unsuitable.
- Makespan on several CPUs: longest-first (LPT) is superior to shortest-first.

9.3 Numerical Comparison

Table 24: Average metrics of all algorithms on the Example 2 workload

Algorithm	AWT	ATAT	ART	Preemptions
FCFS	5.60	9.60	5.60	0
SJF	4.40	8.40	4.40	0
SRTF	4.40	8.40	4.40	0
RR (q = 3)	7.20	11.20	3.80	3
Priority (NP)	4.80	8.80	4.80	0
LJF	7.20	11.20	7.20	0
LRTF	11.80	15.80	3.40	8

Priorities for the Priority algorithm (lower number = higher priority): P1 = 3, P2 = 1, P3 = 4, P4 = 2, P5 = 5. Preemptions = number of times a process is removed from the CPU before it finishes; non-preemptive algorithms have none.

On this workload SJF and SRTF are best on AWT (4.40), FCFS (5.60) and Priority (4.80) are close behind, and LJF (7.20, equal here to Round Robin) is worse than all of them except LRTF (11.80), which however has the best response time (3.40). Round Robin's response time (3.80) is also low, consistent with its design goal.

> **Chapter summary**
> LJF trades waiting time for early service of long jobs; the right algorithm depends on whether the goal is average waiting, response, fairness or makespan.

Chapter 10: Programming Implementations

The following programs implement non-preemptive LJF. They read the number of processes followed by lines of the form `PID AT BT` from standard input and print one row per process in execution order, the Gantt chart, the average metrics, CPU utilisation and throughput. All three were compiled and run on the same inputs and produced identical output. The tie-break rule is the one defined in Section 2.4.

10.1 Python

Listing 1: ljf.py (Python 3.8+)

```
import sys
from dataclasses import dataclass


@dataclass
class Process:
    pid: str
    at: int  # arrival time
    bt: int  # burst time

```

```

def ljf(procs):
    """Non-preemptive Longest Job First. Returns (rows, gantt)."""
    pending = list(enumerate(procs))            # (input index, process)
    t, rows, gantt = 0, [], []
    while pending:
        ready = [(i, p) for i, p in pending if p.at <= t]
        if not ready:                           # CPU idle: jump to next arrival
            nxt = min(p.at for _, p in pending)
            gantt.append(("IDLE", t, nxt))
            t = nxt
            continue
        # longest BT first; ties -> earlier arrival -> smaller input index
        i, p = min(ready, key=lambda x: (-x[1].bt, x[1].at, x[0]))
        pending.remove((i, p))
        st, ct = t, t + p.bt
        rows.append(dict(pid=p.pid, at=p.at, bt=p.bt, st=st, ct=ct,
                         tat=ct - p.at, wt=ct - p.at - p.bt, rt=st - p.at))
        gantt.append((p.pid, st, ct))
        t = ct
    return rows, gantt


def lrtf(procs):
    """Preemptive variant (LRTF), unit-time simulation. Returns gantt."""
    rem = {p.pid: p.bt for p in procs}
    t, cur, gantt = 0, None, []
    while any(rem.values()):
        ready = [p for p in procs if p.at <= t and rem[p.pid] > 0]
        if not ready:
            t, cur = t + 1, None
            continue
        top = max(rem[p.pid] for p in ready)
        cands = [p for p in ready if rem[p.pid] == top]
        nxt = next((p for p in cands if p.pid == cur),
                   min(cands, key=lambda p: p.at))   # tie: keep running one
        if gantt and gantt[-1][0] == nxt.pid and gantt[-1][2] == t:
            gantt[-1] = (nxt.pid, gantt[-1][1], t + 1)
        else:
            gantt.append((nxt.pid, t, t + 1))
        rem[nxt.pid] -= 1
        cur, t = nxt.pid, t + 1
    return gantt


def main():
    tok = sys.stdin.read().split()
    n = int(tok[0]) if tok else 0
    if n == 0:
        print("No processes.")
        return
    procs = [Process(tok[1 + 3 * i], int(tok[2 + 3 * i]), int(tok[3 + 3 * i]))
             for i in range(n)]
    if any(p.at < 0 or p.bt <= 0 for p in procs):
        print("Invalid input: AT must be >= 0 and BT must be > 0")
        sys.exit(1)
    rows, gantt = ljf(procs)
    print(f"{'PID':<5}{'AT':>4}{'BT':>4}{'ST':>4}{'CT':>4}"
          f"{'TAT':>5}{'WT':>4}{'RT':>4}")
    for r in rows:
        print(f"{r['pid']:<5}{r['at']:>4}{r['bt']:>4}{r['st']:>4}{r['ct']:>4}"
              f"{r['tat']:>5}{r['wt']:>4}{r['rt']:>4}")
    print("Gantt: |" + "|".join(f"{g} {a}-{b}" for g, a, b in gantt) + "|")
    end = max(r["ct"] for r in rows)
    busy = sum(r["bt"] for r in rows)
    print(f"AWT={sum(r['wt'] for r in rows) / n:.2f}  "
          f"ATAT={sum(r['tat'] for r in rows) / n:.2f}  "
          f"ART={sum(r['rt'] for r in rows) / n:.2f}")
    print(f"CPU utilization={100.0 * busy / end:.2f}%  Throughput={n / end:.3f}")


if __name__ == "__main__":
    main()
```

Table 25: Line-by-line explanation of the Python program

Lines	Explanation
1–9	Imports and the Process data class with fields pid, at (arrival) and bt (burst).
12–15	Function ljf receives the list; pending holds (input index, process) pairs; t is the clock.
16–22	Ready queue: all pending processes with at <= t. If it is empty the CPU is idle: record an IDLE segment and jump to the next arrival (arrival handling).
23–25	Selection: the minimum of the key (−bt, at, index) is the longest burst, with ties to the earlier arrival and the smaller input index; the chosen process is removed.
26–31	ST = t, CT = ST + BT, TAT = CT − AT, WT = TAT − BT, RT = ST − AT; the clock advances to CT.
34–53	Function lrtf: a separate unit-time simulation of the preemptive variant (not LJF). It keeps the running process on ties.
56–66	Input parsing; n = 0 prints a message; non-positive BT or negative AT is rejected (boundary cases).
67–79	Printing of the table, Gantt chart, averages, CPU utilisation (busy/end × 100) and throughput (n/end).

10.2 C++

Listing 2: ljf.cpp (C++17)

```cpp
#include <algorithm>
#include <cstdio>
#include <iostream>
#include <string>
#include <vector>
using namespace std;

struct Proc {
    string pid;
    int at, bt;
    int st = 0, ct = 0;
    bool done = false;
};
struct Seg { string pid; int a, b; };

int main() {
    int n;
    if (!(cin >> n) || n == 0) { cout << "No processes.\n"; return 0; }
    vector<Proc> p(n);
    for (auto &x : p) {
        cin >> x.pid >> x.at >> x.bt;
        if (x.at < 0 || x.bt <= 0) {
            cout << "Invalid input: AT must be >= 0 and BT must be > 0\n";
            return 1;
        }
    }
    vector<Seg> gantt;
    vector<int> order;
    int t = 0, finished = 0;
    while (finished < n) {
        int best = -1, nextArrival = 1 << 30;
        for (int i = 0; i < n; ++i) {
            if (p[i].done) continue;
            nextArrival = min(nextArrival, p[i].at);
            if (p[i].at > t) continue;              // not in the ready queue
            // longest BT; ties -> earlier arrival -> smaller input index
            if (best < 0 || p[i].bt > p[best].bt ||
                (p[i].bt == p[best].bt && p[i].at < p[best].at))
                best = i;
        }
        if (best < 0) {                             // CPU idle until next arrival
            gantt.push_back({"IDLE", t, nextArrival});
            t = nextArrival;
            continue;
        }
        p[best].st = t;
        t += p[best].bt;
        p[best].ct = t;
        p[best].done = true;
        ++finished;
        order.push_back(best);
        gantt.push_back({p[best].pid, p[best].st, p[best].ct});
```

```
    }
    printf("%-5s%4s%4s%4s%4s%5s%4s%4s\n", "PID", "AT", "BT", "ST", "CT",
           "TAT", "WT", "RT");
    double sw = 0, st = 0, sr = 0, busy = 0;
    for (int i : order) {
        const Proc &x = p[i];
        int tat = x.ct - x.at, wt = tat - x.bt, rt = x.st - x.at;
        printf("%-5s%4d%4d%4d%4d%5d%4d%4d\n", x.pid.c_str(), x.at, x.bt, x.st,
               x.ct, tat, wt, rt);
        sw += wt; st += tat; sr += rt; busy += x.bt;
    }
    printf("Gantt: |");
    for (auto &g : gantt) printf("%s %d-%d|", g.pid.c_str(), g.a, g.b);
    printf("\nAWT=%.2f  ATAT=%.2f  ART=%.2f\n", sw / n, st / n, sr / n);
    printf("CPU utilization=%.2f%%  Throughput=%.3f\n",
           100.0 * busy / t, n / (double)t);
    return 0;
}
```

Table 26: Line-by-line explanation of the C++ program

Lines	Explanation
1–14	Headers and two structures: Proc (data and results) and Seg (a Gantt segment).
16–26	Input: n and the process records; invalid values are rejected and n = 0 prints a message.
27–40	Main loop. For each unfinished process track the earliest pending arrival; processes with at > t are skipped (not in the ready queue); the best candidate has the largest bt, ties to earlier arrival; the strict comparison keeps the smaller index.
41–45	Idle branch: if nothing is ready, record IDLE and set t to the next arrival.
46–53	Dispatch: ST = t; t += BT; CT = t; mark finished; record order and Gantt segment.
54–70	Output: TAT = CT − AT, WT = TAT − BT, RT = ST − AT, then averages, utilisation and throughput.

10.3 Java

Listing 3: LJF.java (Java 11+, run with: java LJF.java)

```
import java.util.*;

public class LJF {
    static class Proc {
        String pid; int at, bt, st, ct; boolean done;
        Proc(String pid, int at, int bt) {
            this.pid = pid; this.at = at; this.bt = bt;
        }
    }

    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.hasNextInt() ? sc.nextInt() : 0;
        if (n == 0) { System.out.println("No processes."); return; }
        Proc[] p = new Proc[n];
        for (int i = 0; i < n; i++) {
            p[i] = new Proc(sc.next(), sc.nextInt(), sc.nextInt());
            if (p[i].at < 0 || p[i].bt <= 0) {
                System.out.println(
                    "Invalid input: AT must be >= 0 and BT must be > 0");
                System.exit(1);
            }
        }
        List<String> gantt = new ArrayList<>();
        List<Integer> order = new ArrayList<>();
        int t = 0, finished = 0;
        while (finished < n) {
            int best = -1, nextArrival = Integer.MAX_VALUE;
            for (int i = 0; i < n; i++) {
                if (p[i].done) continue;
                nextArrival = Math.min(nextArrival, p[i].at);
                if (p[i].at > t) continue;          // not in the ready queue
                // longest BT; ties -> earlier arrival -> smaller input index
                if (best < 0 || p[i].bt > p[best].bt
                        || (p[i].bt == p[best].bt && p[i].at < p[best].at))
                    best = i;
            }
            if (best < 0) {                         // CPU idle until next arrival
                gantt.add("IDLE " + t + "-" + nextArrival);
                t = nextArrival;
                continue;
            }
```

```
43              p[best].st = t;
44              t += p[best].bt;
45              p[best].ct = t;
46              p[best].done = true;
47              finished++;
48              order.add(best);
49              gantt.add(p[best].pid + " " + p[best].st + "-" + p[best].ct);
50          }
51          System.out.println(String.format("%-5s%4s%4s%4s%4s%5s%4s%4s",
52                  "PID", "AT", "BT", "ST", "CT", "TAT", "WT", "RT"));
53          double sw = 0, stt = 0, sr = 0, busy = 0;
54          for (int i : order) {
55              Proc x = p[i];
56              int tat = x.ct - x.at, wt = tat - x.bt, rt = x.st - x.at;
57              System.out.println(String.format("%-5s%4d%4d%4d%4d%5d%4d%4d",
58                      x.pid, x.at, x.bt, x.st, x.ct, tat, wt, rt));
59              sw += wt; stt += tat; sr += rt; busy += x.bt;
60          }
61          System.out.println("Gantt: |" + String.join("|", gantt) + "|");
62          System.out.println(String.format("AWT=%.2f  ATAT=%.2f  ART=%.2f",
63                  sw / n, stt / n, sr / n));
64          System.out.println(String.format("CPU utilization=%.2f%%  Throughput=%.3f",
65                  100.0 * busy / t, n / (double) t));
66      }
67  }
```

Table 27: Line-by-line explanation of the Java program

Lines	Explanation
1–9	Process class with fields for identifier, AT, BT and the computed ST and CT.
11–23	Reading n and the processes with Scanner; invalid values or n = 0 are handled before scheduling.
24–37	Same selection loop as in C++: skip processes that have not arrived, choose the largest BT, ties to earlier arrival and smaller index.
38–42	Idle branch: add an IDLE entry and advance the clock.
43–50	Dispatch and completion: compute ST and CT, update the order list and the Gantt list.
51–66	Formatted output of rows, Gantt chart and the summary metrics.

10.4 Sample Input and Output

Sample input (Example 2)

```
5
P1 0 4
P2 1 3
P3 2 6
P4 3 2
P5 5 5
```

Sample output (identical in Python, C++ and Java)

```
PID    AT  BT  ST  CT  TAT  WT  RT
P1      0   4   0   4    4   0   0
P3      2   6   4  10    8   2   2
P5      5   5  10  15   10   5   5
P2      1   3  15  18   17  14  14
P4      3   2  18  20   17  15  15
Gantt: |P1 0-4|P3 4-10|P5 10-15|P2 15-18|P4 18-20|
AWT=7.20  ATAT=11.20  ART=7.20
CPU utilization=100.00%  Throughput=0.250
```

The output reproduces Table 10 and Figure 4: the order P1, P3, P5, P2, P4, the waiting times 0, 2, 5, 14 and 15, and an average waiting time of 7.20.

10.5 Boundary Cases and Verification

Input with an idle gap (P1: AT 0, BT 3; P2: AT 10, BT 4) and its output

```
2
P1 0 3
P2 10 4

PID    AT  BT  ST  CT  TAT  WT  RT
P1      0   3   0   3    3   0   0
P2     10   4  10  14    4   0   0
Gantt: |P1 0-3|IDLE 3-10|P2 10-14|
AWT=0.00  ATAT=3.50  ART=0.00
CPU utilization=50.00%  Throughput=0.143
```

- Empty input (n = 0): the program prints 'No processes.' and stops.
- Invalid data (BT ≤ 0 or AT < 0): rejected with a message and exit code 1.
- Idle CPU: the clock jumps to the next arrival, and the Gantt chart shows an IDLE segment (utilisation 50.00% above).
- First arrival after 0: a single process arriving at 7 produces IDLE 0–7 and zero waiting time.
- Ties: resolved by earlier arrival then input order, as in Example 3.
- Rounding: all values are printed with fixed decimals; exact binary ties in the last digit may round differently between languages, which is why throughput is printed with three decimals.

Table 28: Verification of the implementations on the article's workloads

Input	AWT	ATAT	Utilisation	Identical in 3 languages	Matches simulator
Example 1	16.20	22.60	100.00%	Yes	Yes
Example 2	7.20	11.20	100.00%	Yes	Yes
Example 3	14.00	19.83	100.00%	Yes	Yes
Example 4	26.40	31.90	100.00%	Yes	Yes
Idle gap	0.00	3.50	50.00%	Yes	Yes
Single, AT = 7	0.00	4.00	36.36%	Yes	Yes

The Python implementation was also compared segment by segment with the reference simulator for Examples 1–4 and the idle-gap workload, and its LRTF function was checked against the Example 5 schedule.

Chapter summary

Three independent programs agree on every test, including boundary cases, and agree with the hand-solved examples.

Chapter 11: Applications and Case Studies

This chapter distinguishes between theoretical applicability, where the policy is a valid model or heuristic, and practical use in real operating systems, where it is rare. Claims are tied to the cited literature.

11.1 Theoretical Use versus Practice in Real Operating Systems

Mainstream textbooks present FCFS, SJF/SRTF, priority and Round Robin, together with multilevel queues, as the basic repertoire, and note that SJF cannot be implemented exactly at short-term level because the next burst is unknown [1, 2, 3, 9]. Longest-first rules are not part of this standard repertoire. Production general-purpose schedulers use priority and fairness mechanisms rather than burst length, so LJF should be viewed as a model and a heuristic, not as a policy that real kernels adopt as their main algorithm.

11.2 Batch Processing and Offline Systems

In batch and offline processing all jobs are known in advance, there is no interactivity, and the goal is often to finish a set of jobs as early as possible on several workers. Here longest-first is the LPT rule and is the standard simple heuristic for minimising makespan, with the $4/3 - 1/(3m)$ guarantee [7, 8]. For a single worker with the goal of minimising mean completion time, shortest-first is optimal [5], so LJF is the wrong choice.

Table 29: Suitability of LJF in different environments

Environment	Main objective	LJF suitability	Comment
Batch (single CPU)	Mean waiting time	Poor	SJF is optimal [5]
Batch (parallel machines)	Makespan	Good heuristic	LPT guarantee [7]
Offline processing	Throughput of whole set	Fair	Order matters only for waiting
Educational simulators	Illustration	Very good	Contrasts with SJF
Long-running computations	Balanced finish	Good in parallel	Needs reliable estimates
Interactive systems	Response time	Unsuitable	Unbounded delay

11.3 Educational Simulation Environments

Process-scheduling simulators and course assignments commonly include LJF because it can be implemented in a few lines and gives results that contradict intuition: the algorithm that serves the biggest job first has the worst average waiting time. This makes it an effective example for discussing objectives, optimality and starvation.

11.4 Environments with Long-Running Tasks

When tasks run for hours and cannot be preempted cheaply (for example, large simulations or training jobs), preemption is costly and a non-preemptive rule is natural. Starting the longest tasks first can shorten the completion time of the whole batch on a cluster, but it should be combined with estimates of run time and with a policy that keeps short tasks from waiting indefinitely (Chapter 12).

Chapter summary

LJF is well justified as a makespan heuristic for known workloads on parallel resources and as a teaching tool, and poorly justified as a general-purpose single-CPU policy.

Chapter 12: Improvements and Alternative Approaches

The techniques below are not part of the classical LJF algorithm. They are complementary or alternative methods that address its weaknesses.

12.1 Aging

Aging gradually improves the rank of waiting processes. A simple LJF variant ranks by the score $s_i = BT_i + beta\ (t - AT_i)$, with $beta > 0$. A job with burst b that has waited W outranks any newly arrived job of burst at most B_max once $b + beta\ W > B_max$, i.e. after at most $(B_max - b)/beta$ units. Hence no ready job waits indefinitely (beyond the current job's completion) and starvation disappears. With $beta = 0$ the classical LJF is recovered.

12.2 Starvation Prevention

Besides aging, starvation can be prevented by a bounded-wait guarantee (a job waiting longer than a threshold is promoted), by reserving a share of CPU time for short jobs, or by serving a long job only if the queue length is below a limit.

12.3 Hybrid Scheduling

A hybrid policy uses LJF for batch classes and a different policy for interactive classes, for example two queues with Round Robin for short, interactive work and LJF for the background batch queue, with the CPU divided between queues.

12.4 Dynamic Priority

Dynamic priority recomputes ranks at runtime from waiting time, estimated length or consumed CPU time. Aging is the simplest instance; the Highest Response Ratio Next rule, with ratio (W + BT)/BT, is another, and favours short jobs while still rewarding waiting [2].

12.5 Combining LJF and Priority Scheduling

A lexicographic combination ranks by the external priority first and uses LJF only among processes of equal priority. This preserves administrative control (important jobs go first) while retaining the makespan benefit within each priority class.

12.6 Better Handling of Short Processes

Short processes can be protected by a separate fast lane, by switching to SJF when the queue length exceeds a threshold, or by splitting long jobs into smaller chunks that can interleave with short ones.

12.7 Fairness-Oriented Alternatives

Table 30: Methods that complement or replace LJF

Method	Idea	Addresses	Cost
Aging	Add waiting time to the rank	Starvation	Rank recomputation
Bounded-wait promotion	Promote after a limit	Starvation	Needs thresholds
Hybrid queues	Different policy per class	Interactivity	Two schedulers
HRRN	Rank (W + BT)/BT	Fairness	Needs BT
Priority then LJF	Priority first, LJF as tie-break	Control	Priority assignment
Round Robin	Time slices	Response, fairness	Higher AWT

Chapter summary

Aging, hybrid queues and dynamic priority remove LJF's starvation problem at the cost of departing from the classical algorithm.

Chapter 13: Summary and Conclusions

13.1 Review of Main Concepts

CPU scheduling selects the next process from the ready queue. LJF selects the largest burst time; the classical form is non-preemptive, whereas its preemptive relatives LJF-PA and LRTF differ in the ranking key and the instants of decision. LJF and LRTF are different algorithms.

13.2 Summary of the Algorithm and Numerical Findings

Table 31: Summary of numerical results (LJF unless stated, with SJF AWT for reference)

Workload	n	Feature	AWT	ATAT	SJF AWT
Example 1	5	All AT = 0	16.20	22.60	9.40
Example 2	5	Staggered AT	7.20	11.20	4.40
Example 3	6	Equal BT	14.00	19.83	9.00
Example 4	10	Queue build-up	26.40	31.90	13.40
Example 5	4	LJF (NP)	6.25	11.25	4.25

In every example LJF has a higher average waiting time than SJF on the same workload, and the gap is largest when many short jobs coexist with long ones.

13.3 Evaluation of Strengths and Weaknesses

Strengths: simplicity, early service of long jobs, analysability and usefulness as a makespan heuristic. Weaknesses: poor average waiting time, starvation of short jobs, dependence on burst estimates and unsuitability for interactive use.

13.4 Comparison with Other Algorithms

LJF is dominated by SJF and SRTF with respect to average waiting and turnaround time, by Round Robin with respect to response and fairness, and by priority schemes with aging with respect to control. Its niche is longest-first load balancing.

13.5 Limitations of the Algorithm and of This Study

The analysis assumes that burst times are known exactly, that processes use the CPU for a single burst, and that context-switch and I/O costs are zero. Examples are small and deterministic. Real workloads, multi-core effects and burst prediction errors are outside the scope.

13.6 Suggestions for Future Research

- Evaluate LJF with predicted rather than exact burst times and quantify the effect of estimation errors.
- Study aging parameters that bound waiting time while preserving the makespan benefit.
- Extend the analysis to multi-processor schedulers and to workloads with I/O bursts.
- Compare LJF, LRTF and fairness-oriented policies on empirical workload traces.

Chapter 14: References

[1] A. Silberschatz, P. B. Galvin and G. Gagne, Operating System Concepts, 10th ed. Hoboken, NJ: Wiley, 2018.

[2] W. Stallings, Operating Systems: Internals and Design Principles, 9th ed. Harlow: Pearson, 2017.

[3] A. S. Tanenbaum and H. Bos, Modern Operating Systems, 4th ed. Boston, MA: Pearson, 2015.

[4] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA: MIT Press, 2022.

[5] W. E. Smith, “Various optimizers for single-stage production,” Naval Research Logistics Quarterly, vol. 3, no. 1–2, pp. 59–66, 1956. doi: 10.1002/nav.3800030106.

[6] R. L. Graham, E. L. Lawler, J. K. Lenstra and A. H. G. Rinnooy Kan, “Optimization and approximation in deterministic sequencing and scheduling: a survey,” Annals of Discrete Mathematics, vol. 5, pp. 287–326, 1979. doi: 10.1016/S0167-5060(08)70356-X.

[7] R. L. Graham, “Bounds on multiprocessor timing anomalies,” SIAM Journal on Applied Mathematics, vol. 17, no. 2, pp. 416–429, 1969. doi: 10.1137/0117039.

[8] M. L. Pinedo, Scheduling: Theory, Algorithms, and Systems, 5th ed. Cham: Springer, 2016. doi: 10.1007/978-3-319-26580-3.

[9] R. H. Arpaci-Dusseau and A. C. Arpaci-Dusseau, Operating Systems: Three Easy Pieces, Arpaci-Dusseau Books, 2018. [Online]. Available: https://pages.cs.wisc.edu/~remzi/OSTEP/

Note: the textbooks above present the classical algorithms (FCFS, SJF, SRTF, Round Robin, priority) and the evaluation criteria used here; the specific statements about LJF and LRTF in this article are derived by the author and verified numerically, since these policies are not standard chapters of those books.